Name:R.K Ayushman Singh

Reg no:261100690014

Cybersecurity Log Analysis Using Linux Tools

1. Aim

To analyse cybersecurity log records using Linux command-line tools such as `grep`, `sed`, `awk`, Bash scripting, and pipelines to identify suspicious login activities.

2. Introduction

Log analysis is an important part of cybersecurity because it helps administrators monitor system activities and identify possible security threats.

For this experiment, I selected SSH login monitoring. I created a sample authentication log containing both successful and failed login attempts from different IP addresses. I then used Linux commands to filter the records, extract IP addresses, count repeated login attempts, and identify suspicious activities.

The main purpose was to understand how basic Linux tools can be combined to perform simple security log analysis.

3. Selected Cybersecurity Scenario

Scenario: Detection of Repeated SSH Login Attempts

SSH (Secure Shell) is used to access remote systems securely. Sometimes, attackers try different usernames and passwords repeatedly to gain unauthorized access. This activity is commonly known as a brute-force attack.

If someone continuously tries to log in using different passwords, the failed attempts are recorded in the SSH authentication log. By analysing these records, we can identify IP addresses that generate an unusually high number of failed login attempts.

For this experiment, I created a simulated SSH authentication log containing successful and unsuccessful login attempts from different IP addresses.

4. Log File Acquisition

The log file was created manually for an authorized laboratory simulation on my personal Linux system.

Details	Description
Environment	Personal Linux system
Operating system	Garuda Linux
Log file	auth.log
Simulation method	Manually created SSH authentication records
Tools used	Nano, Bash, grep, sed and awk
Dataset source	Self-created simulated data
Creation date	5 November 2026

First, I created a separate directory for the experiment.

Commands used:

```
anonymous@anonymous-supreme in ~ via v3.14.7
❯ mkdir -p ~/log-analysis-assignment

anonymous@anonymous-supreme in ~ via v3.14.7
❯ cd ~/log-analysis-assignment

anonymous@anonymous-supreme in ~/log-analysis-assignment via v3.14.7
❯ mkdir output

anonymous@anonymous-supreme in ~/log-analysis-assignment via v3.14.7
❯ touch auth.log

anonymous@anonymous-supreme in ~/log-analysis-assignment via v3.14.7
❯ touch analyze.awk

anonymous@anonymous-supreme in ~/log-analysis-assignment via v3.14.7
❯ touch analyze.sh

anonymous@anonymous-supreme in ~/log-analysis-assignment via v3.14.7
❯ touch README.txt

anonymous@anonymous-supreme in ~/log-analysis-assignment via v3.14.7
❯ _
```

I checked whether the files were created successfully.

```
ls -l
```

After that, I opened the log file using Nano.

```
nano auth.log
```

I manually entered sample SSH authentication records containing failed login attempts, successful logins, different usernames, and different IP addresses.

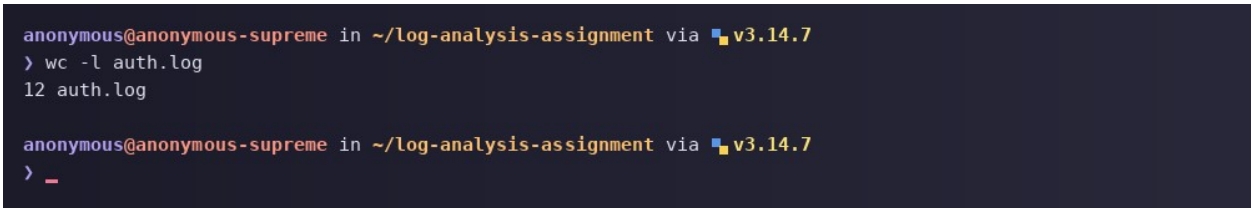
The file was saved using `Ctrl + O`, followed by `Enter`, and then `Ctrl + X`.

To verify the records, I used:

```
cat auth.log
```

To check the total number of records:

Output:



No real system was accessed or tested. All login records and IP addresses used in this experiment are sample data.

5. Log File Structure

The simulated log file follows a format similar to Linux SSH authentication logs.

Example log record:

```
Nov  5 10:01:02 lab-pc sshd[1201]: Failed password for invalid user admin
from 192.168.1.50 port 50122 ssh2
```

The important fields in this record are:

Field	Description
Date	Date of the login activity
Time	Time at which the event occurred
Hostname	Name of the system
Process	SSH service (`sshd`)
Process ID	Identification number of the process
Event	Failed or successful login
Username	Account used during login
IP address	Source address of the login attempt
Port	Source port number
Protocol	SSH protocol version

To inspect the log file, I used:

```
cat auth.log
```

I also used the following command to display only the first five records:

```
head -n 5 auth.log
```

These commands helped me understand the structure and contents of the log file before starting the analysis.

6. Cybersecurity Investigation Questions

Before analysing the records, I decided to investigate the following questions:

1. Which IP address generated the highest number of failed login attempts?
2. Are there any IP addresses generating repeated login failures?
3. Were there any successful login attempts in the log file?
4. Did an IP address generate multiple failed attempts before a successful login?

These questions helped me decide which information needed to be extracted from the log file.

7. Tools and Commands Used

I used different Linux commands to filter and analyse the authentication records.

A. grep – Filtering log records

To display only failed login attempts:

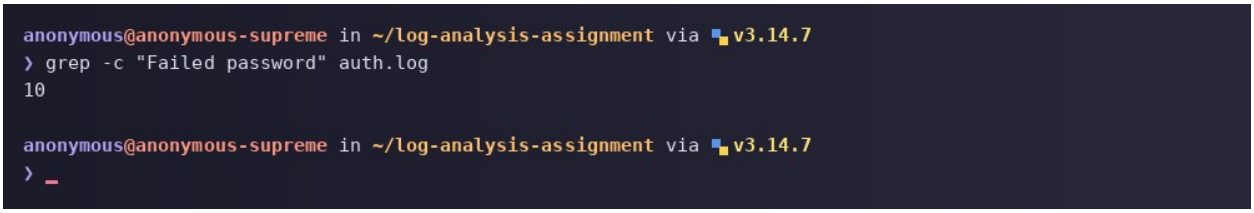
```
grep "Failed password" auth.log
```

To display successful login attempts:

```
grep "Accepted password" auth.log
```

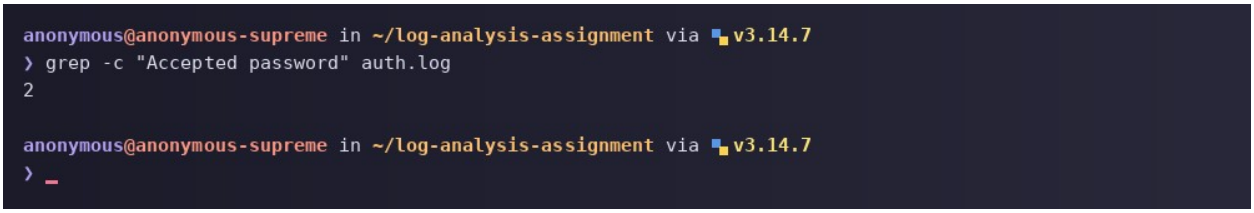
To count the failed login attempts:

Output:



To count successful login attempts:

Output:



B. sed – Cleaning log records

I used `sed` to replace multiple spaces with a single space.

```
sed -E 's/[[:space:]]+/ /g' auth.log
```

This made the log records more consistent and easier to process.

C. awk – Extracting IP addresses

I used AWK to extract the IP addresses from the failed login records.

First, I filtered the failed login attempts and then extracted the IP addresses:

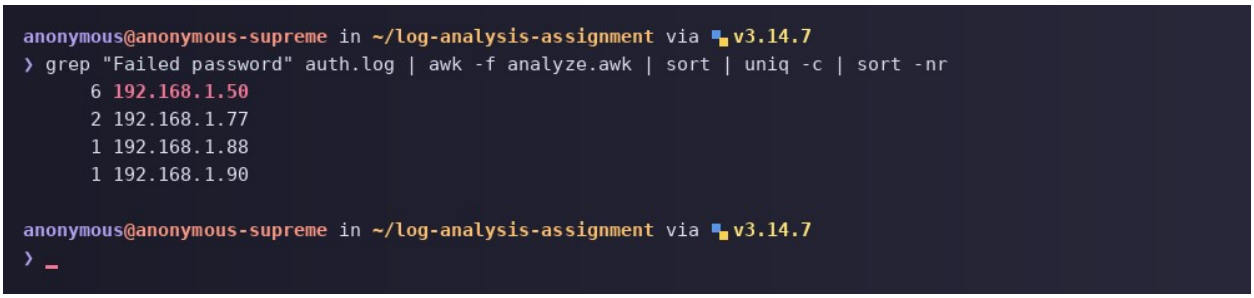
```
grep "Failed password" auth.log | awk -f analyze.awk
```

The AWK script searches for the word `from` and prints the IP address appearing immediately after it.

D. sort and uniq – Counting repeated IP addresses

To count the number of failed login attempts from each IP address, I used:

Output:



This output shows that `192.168.1.50` generated the highest number of failed login attempts.

E. Other commands

I also used the following commands during the experiment:

Command	Purpose
`mkdir`	Creates directories
`touch`	Creates empty files
`nano`	Opens files for editing
`cat`	Displays file contents
`head -n 5`	Displays the first five records
`wc -l`	Counts the number of records
`chmod +x`	Gives execution permission to the Bash script
`ls -l`	Displays files and their permissions
`find`	Displays the project directory contents

Power filters and pipelines

I combined multiple commands using the pipe operator (`|`) so that the output of one command could be used as input for another command.

For example:

```
grep "Failed password" auth.log | awk -f analyze.awk | sort | uniq -c | sort -nr
```

This command performs the following operations:

1. `grep` selects failed login records.
2. `awk` extracts IP addresses.
3. `sort` arranges the IP addresses.
4. `uniq -c` counts repeated IP addresses.
5. `sort -nr` arranges the counts in descending order.

This helped me perform the analysis without manually checking every record.

8. Bash Script Implementation

After testing the commands individually, I combined them into a single Bash script to automate the analysis.

I created the script using:

```
nano analyze.sh
```

The script was designed to check the log file, clean the records, separate successful and failed logins, count IP addresses, and identify suspicious activity.

The following is the Bash script used:

GNU nano 9.2 analyze.sh

```
#!/bin/bash

# SSH Log Analysis Script

LOG_FILE="auth.log"
OUTPUT_DIR="output"
THRESHOLD=5

# Check whether log file exists
if [ ! -f "$LOG_FILE" ]; then
    echo "Error: Log file not found!"
    exit 1
fi

# Create output directory
mkdir -p "$OUTPUT_DIR"

echo "================================="
echo "     SSH LOG ANALYSIS REPORT"
echo "================================="

# Normalize log file using sed
sed -E 's/[[:space:]]+/ /g' "$LOG_FILE" \
> "$OUTPUT_DIR/normalized.log"

# Extract failed login records
grep "Failed password" "$OUTPUT_DIR/normalized.log" \
> "$OUTPUT_DIR/failed.log"

# Extract successful login records
```

```
GNU nano 9.2                                    analyze.sh
grep "Accepted password" "$OUTPUT_DIR/normalized.log" \
> "$OUTPUT_DIR/successful.log"

# Count total records
TOTAL=$(wc -l < "$LOG_FILE")

# Count failed and successful attempts
FAILED=$(wc -l < "$OUTPUT_DIR/failed.log")
SUCCESS=$(wc -l < "$OUTPUT_DIR/successful.log")

echo ""
echo "Total log records: $TOTAL"
echo "Failed login attempts: $FAILED"
echo "Successful login attempts: $SUCCESS"

# Extract IP addresses and count occurrences
awk -f analyze.awk "$OUTPUT_DIR/failed.log" |
sort |
uniq -c |
sort -nr > "$OUTPUT_DIR/ip_counts.txt"

echo ""
echo "Failed login attempts by IP:"
cat "$OUTPUT_DIR/ip_counts.txt"

# Identify suspicious IP addresses
echo ""
echo "Suspicious IP addresses (5 or more failures):"

awk -v limit="$THRESHOLD" '
```

```
GNU nano 9.2                                    analyze.sh
{
    if ($1 >= limit) {
        print $2, "- Failed attempts:", $1
        found = 1
    }
}
END {
    if (!found)
        print "No suspicious IP addresses found."
}
' "$OUTPUT_DIR/ip_counts.txt" \
> "$OUTPUT_DIR/suspicious.txt"

cat "$OUTPUT_DIR/suspicious.txt"

# Display successful login records
echo ""
echo "Successful login records:"
cat "$OUTPUT_DIR/successful.log"

echo ""
echo "Analysis completed successfully."
echo "Results are saved in the output directory."



^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify
```

I saved the script and gave it execution permission.

```
chmod +x analyze.sh
```

To execute the script, I used:

```
./analyze.sh
```

The script automatically performed all the analysis steps and generated separate output files inside the `output` directory.

Purpose of the main script commands

Command	Purpose
`if [ ! -f "$LOG_FILE" ]`	Checks whether the log file exists
`mkdir -p`	Creates the output directory if needed
`sed`	Normalizes spaces in log records
`grep`	Separates failed and successful login records
`wc -l`	Counts the records
`awk -f`	Executes the separate AWK script
`sort`	Sorts the extracted IP addresses
`uniq -c`	Counts repeated IP addresses
`awk -v limit`	Checks whether an IP has reached the suspicious activity threshold
`cat`	Displays the generated results

9. Analysis and Observations

After executing the Bash script, I obtained the following results.

Command used:

```
./analyze.sh
```

Output:

```
anonymous@anonymous-supreme in ~/log-analysis-assignment via v3.14.7
❯ ./analyze.sh
==================================
     SSH LOG ANALYSIS REPORT
==================================

Total log records: 12
Failed login attempts: 10
Successful login attempts: 2

Failed login attempts by IP:
      6 192.168.1.50
      2 192.168.1.77
      1 192.168.1.88
      1 192.168.1.90

Suspicious IP addresses (5 or more failures):
192.168.1.50 - Failed attempts: 6

Successful login records:
Nov 5 10:05:20 lab-pc sshd[1210]: Accepted password for student from 192.168.1.50 port 50401 ssh2
Nov 5 10:06:10 lab-pc sshd[1211]: Accepted password for student from 192.168.1.25 port 50402 ssh2

Analysis completed successfully.
Results are saved in the output directory.

anonymous@anonymous-supreme in ~/log-analysis-assignment via v3.14.7
❯
```

The results showed that there were 12 total records, including 10 failed login attempts and 2 successful login attempts.

IP address analysis:

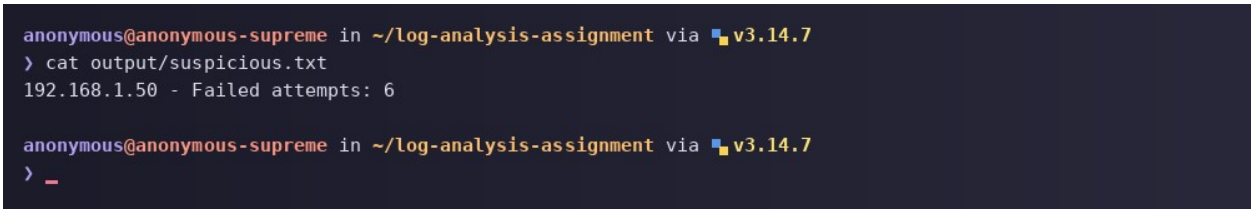
IP address	Failed attempts
192.168.1.50	6
192.168.1.77	2
192.168.1.88	1
192.168.1.90	1

The IP address `192.168.1.50` generated six failed login attempts, which was higher than the other addresses.

I also observed that the same IP address had a successful login record later in the log. This pattern could indicate a possible password-guessing attempt followed by a successful login. However, it cannot be confirmed as an attack from the available sample records alone.

To view the suspicious IP report separately, I used:

Output:



To view the IP address counts:

```
cat output/ip_counts.txt
```

To view the successful login records:

```
cat output/successful.log
```

These files helped me check the results without running the complete analysis again.

10. Suspicious Activity Detection and Threshold

For this experiment, I selected a threshold of five failed login attempts from the same IP address.

An IP address generating five or more failed login attempts was classified as suspicious.

The threshold was selected to identify repeated login failures instead of treating every failed login as a possible attack.

Based on the analysis:

- `192.168.1.50` – Suspicious (6 failed attempts).
- `192.168.1.77` – Not flagged (2 failed attempts).
- `192.168.1.88` – Not flagged (1 failed attempt).
- `192.168.1.90` – Not flagged (1 failed attempt).

The threshold is only a basic detection rule for this experiment. In a real environment, the number of failed attempts should also be considered along with the time interval, account details, and normal login behaviour.

11. False Positives and Limitations

False Positives

A false positive occurs when normal activity is incorrectly identified as suspicious.

For example, a genuine user may enter an incorrect password several times. If the number of failed attempts reaches the selected threshold, the IP address may be flagged even though no attack has taken place.

Also, multiple users connected through the same network may share an IP address. Their login failures could be counted together.

Limitations

1. The experiment uses simulated data instead of real authentication logs.
2. The analysis uses a fixed threshold of five failed attempts.
3. It does not analyse login attempts over different time intervals.
4. It does not identify the actual person responsible for an IP address.
5. It cannot confirm whether a successful login was unauthorized.
6. Attacks distributed across different IP addresses may not be detected by this simple method.

Therefore, the results should be treated as indicators of suspicious activity rather than proof of an attack.

12. Result

The SSH authentication log was successfully analysed using Linux command-line tools and Bash scripting.

The experiment identified one suspicious IP address, `192.168.1.50`, with six failed login attempts.

The script also separated successful and failed login records, counted the attempts from different IP addresses, and generated separate output files for further inspection.

Thus, the experiment demonstrated how Linux filtering commands, pipelines, sed, awk, and Bash scripting can be combined to perform basic cybersecurity log analysis.

13. Conclusion

Through this experiment, I learned how to analyse authentication logs using Linux tools.

I first created a sample SSH authentication log and tested different commands such as grep, sed, awk, sort, and uniq. After understanding their individual functions, I combined them into a Bash script to automate the analysis.

The experiment helped me understand how repeated login failures can be identified using command-line tools. I also learned how Bash scripting can reduce manual work by processing log files and generating reports automatically.